03 - P2P 네트워킹 시스템

개요

Go-Ethereum의 P2P 네트워킹 시스템은 분산 이더리움 네트워크에서 노드 간 통신을 담당하는 핵심 컴포넌트입니다. 이 시스템은 피어 발견, 연결 관리, 프로토콜 핸들링, 데이터 동기화 등의 기능을 제공하여 블록체인 네트워크의 분산성과 신뢰성을 보장합니다.

P2P 시스템은 다음과 같은 주요 구성 요소로 이루어져 있습니다:

- **p2p.Server**: 전체 P2P 서버 관리 및 피어 연결 처리
- **Discovery 시스템**: UDPv4/v5를 통한 피어 발견 메커니즘
- **Protocol Handler**: 이더리움 프로토콜 메시지 처리
- **Downloader**: 블록체인 데이터 동기화

핵심 구조체 및 인터페이스

p2p.Server 구조체

`p2p.Server` 는 P2P 네트워킹의 중심 구조체로, 모든 피어 연결과 프로토콜을 관리합니다.

```
type Server struct {

// Config fields may not be modified while the server is running.

Config



// Hooks for testing

newTransport  func(net.Conn, *ecdsa.PublicKey) transport
```

```
newPeerHook  func(*Peer)
listenFunc  func(network, addr  string) (net.Listener, error)


lock sync.Mutex // protects running
running  bool


listener net.Listener
ourHandshake *protoHandshake
loopWG sync.WaitGroup
peerFeed event.Feed
log log.Logger


nodedb *enode.DB
localnode *enode.LocalNode
discv4 *discover.UDPv4
discv5 *discover.UDPv5
discmix *enode.FairMix
dialsched *dialScheduler


// Channels into the run loop
quit  chan  struct{}
addtrusted  chan *enode.Node
removetrusted  chan *enode.Node
```

```
peerOp  chan peerOpFunc
peerOpDone  chan  struct{}
delpeer  chan peerDrop
checkpointPostHandshake  chan *conn
checkpointAddPeer  chan *conn

// State of run loop and listenLoop
inboundHistory expHeap
}
```

p2p.Peer 구조체

개별 피어 연결을 나타내는 구조체입니다.

```
type Peer struct {
rw *conn
running  map[string]*protoRW
log log.Logger
created mclock.AbsTime

wg sync.WaitGroup
protoErr  chan  error
closed  chan  struct{}
pingRecv  chan  struct{}
```

```
disc  chan DiscReason


events *event.Feed

testPipe *MsgPipeRW

}
```

주요 프로세스

1. P2P 서버 시작 과정

Server.Start() 메서드

```
func (srv *Server) Start() (err  error) {

srv.lock.Lock()

defer  srv.lock.Unlock()

if  srv.running {

return  errors.New("server already running")

}

srv.running = true

// 로컬 노드 설정

if  err := srv.setupLocalNode();  err != nil {

return  err

}

// 포트 매핑 설정
```

```go
srv.setupPortMapping()


// TCP 리스너 설정
if srv.ListenAddr != "" {
if err := srv.setupListening(); err != nil {
return err
}
}
// 디스커버리 설정
if err := srv.setupDiscovery(); err != nil {
return err
}
// 다이얼 스케줄러 설정
srv.setupDialScheduler()


// 메인 루프 시작
srv.loopWG.Add(1)
go srv.run()
return nil
}
```

로컬 노드 설정

```go
func (srv *Server) setupLocalNode() error {
// devp2p 핸드셰이크 생성
pubkey := crypto.FromECDSAPub(&srv.PrivateKey.PublicKey)
srv.ourHandshake = &protoHandshake{
Version: baseProtocolVersion,
Name: srv.Name,
ID: pubkey[1:],
}
// 프로토콜 capabilities 추가
for _, p := range srv.Protocols {
srv.ourHandshake.Caps = append(srv.ourHandshake.Caps, p.cap())
}
// 로컬 노드 생성
db, err := enode.OpenDB(srv.NodeDatabase)
if err != nil {
return err
}
srv.nodedb = db
srv.localnode = enode.NewLocalNode(db, srv.PrivateKey)
srv.localnode.SetFallbackIP(net.IP{127, 0, 0, 1})
return nil
}
```

2. 피어 발견 메커니즘

UDPv4 디스커버리

`discover.UDPv4` 구조체는 Kademlia 기반의 피어 발견을 구현합니다.

```
type UDPv4 struct {
conn UDPConn
log log.Logger
netrestrict *netutil.Netlist
priv *ecdsa.PrivateKey
localnode *enode.LocalNode
db *enode.DB
tab *Table
closeOnce sync.Once
wg sync.WaitGroup

addReplyMatcher  chan *replyMatcher
gotreply  chan reply
closeCtx context.Context
cancelCloseCtx context.CancelFunc
}
```

피어 발견 과정

1. **PING/PONG 메시지 교환**:

```
func (t *UDPv4) ping(n *enode.Node) (seq  uint64, err  error) {
addr, ok := n.UDPEndpoint()
if !ok {
return  0, errNoUDPEndpoint
}
rm := t.sendPing(n.ID(), addr, nil)
if  err = <-rm.errc;  err == nil {
seq = rm.reply.(*v4wire.Pong).ENRSeq
}
return  seq, err
}
```

2. **FINDNODE 요청**:

```
func (t *UDPv4) findnode(toid enode.ID, toAddrPort netip.AddrPort, target v4wire
t.ensureBond(toid, toAddrPort)

nodes := make([]*enode.Node, 0, bucketSize)
nreceived := 0
rm := t.pending(toid, toAddrPort.Addr(), v4wire.NeighborsPacket, func(r v4wire.P
reply := r.(*v4wire.Neighbors)
for  _, rn := range  reply.Nodes {
nreceived++
```

```
n, err := t.nodeFromRPC(toAddrPort, rn)
if  err != nil {
t.log.Trace("Invalid neighbor node received", "ip", rn.IP, "addr", toAddrPort, "
continue
}
nodes = append(nodes, n)
}
return true, nreceived >= bucketSize
})
t.send(toAddrPort, toid, &v4wire.Findnode{
Target: target,
Expiration: uint64(time.Now().Add(expiration).Unix()),
})
err := <-rm.errc
if  errors.Is(err, errTimeout) && rm.reply != nil {
err = nil
}
return  nodes, err
}
```

UDPv5 디스커버리

UDPv5는 더 향상된 보안과 기능을 제공합니다.

```go
type UDPv5 struct {
conn UDPConn
tab *Table
netrestrict *netutil.Netlist
priv *ecdsa.PrivateKey
localNode *enode.LocalNode
db *enode.DB
log log.Logger
clock mclock.Clock
validSchemes enr.IdentityScheme
respTimeout time.Duration


talk *talkSystem


packetInCh  chan ReadPacket
readNextCh  chan  struct{}
callCh  chan *callV5
callDoneCh  chan *callV5
respTimeoutCh  chan *callTimeout
sendCh  chan sendRequest
unhandled  chan<- ReadPacket
```

```
codec codecV5

activeCallByNode  map[enode.ID]*callV5

activeCallByAuth  map[v5wire.Nonce]*callV5

callQueue  map[enode.ID][]*callV5



closeOnce sync.Once

closeCtx context.Context

cancelCloseCtx context.CancelFunc

wg sync.WaitGroup

}
```

3. 피어 연결 관리

연결 설정 과정

```
func (srv *Server) SetupConn(fd net.Conn, flags connFlag, dialDest *enode.Node)

c := &conn{fd: fd, flags: flags, cont: make(chan  error)}

if  dialDest == nil {

c.transport = srv.newTransport(fd, nil)

} else {

c.transport = srv.newTransport(fd, dialDest.Pubkey())

}



err := srv.setupConn(c, dialDest)
```

```
if err != nil {
if !c.is(inboundConn) {
markDialError(err)
} else {
markServeError(err)
}
c.close(err)
}
return err
}
```

핸드셰이크 과정

```
func (srv *Server) setupConn(c *conn, dialDest *enode.Node) error {
// RLPx 핸드셰이크 실행
remotePubkey, err := c.doEncHandshake(srv.PrivateKey)
if err != nil {
srv.log.Trace("Failed RLPx handshake", "addr", c.fd.RemoteAddr(), "conn", c.flag
return fmt.Errorf("%w: %v", errEncHandshakeError, err)
}
if dialDest != nil {
c.node = dialDest
} else {
c.node = nodeFromConn(remotePubkey, c.fd)
```

```
}
// 핸드셰이크 후 검증
err = srv.checkpoint(c, srv.checkpointPostHandshake)
if err != nil {
return err
}

// 프로토콜 핸드셰이크 실행
phs, err := c.doProtoHandshake(srv.ourHandshake)
if err != nil {
return &protoHandshakeError{err: err}
}
c.caps, c.name = phs.Caps, phs.Name
err = srv.checkpoint(c, srv.checkpointAddPeer)
if err != nil {
return err
}

return nil
}
```

4. 프로토콜 핸들링

eth.handler 구조체

이더리움 프로토콜 메시지를 처리하는 핸들러입니다.

```
type ethHandler handler

func (h *ethHandler) Chain() *core.BlockChain { return h.chain }
func (h *ethHandler) TxPool() eth.TxPool { return h.txpool }

func (h *ethHandler) RunPeer(peer *eth.Peer, hand eth.Handler) error {
return (*handler)(h).runEthPeer(peer, hand)
}
```

메시지 처리

```
func (h *ethHandler) Handle(peer *eth.Peer, packet eth.Packet) error {
switch packet := packet.(type) {
case *eth.NewPooledTransactionHashesPacket:
return h.txFetcher.Notify(peer.ID(), packet.Types, packet.Sizes, packet.Hashes)

case *eth.TransactionsPacket:
for _, tx := range *packet {
if tx.Type() == types.BlobTxType {
return errors.New("disallowed broadcast blob transaction")
}
```

```
}
return h.txFetcher.Enqueue(peer.ID(), *packet, false)

case *eth.PooledTransactionsResponse:
// 블롭 트랜잭션 검증
for _, tx := range *packet {
if tx.Type() == types.BlobTxType {
if tx.BlobTxSidecar() == nil {
return errors.New("received sidecar-less blob transaction")
}
if err := tx.BlobTxSidecar().ValidateBlobCommitmentHashes(tx.BlobHashes()); er
return err
}
}
}
return h.txFetcher.Enqueue(peer.ID(), *packet, true)

default:
return fmt.Errorf("unexpected eth packet type: %T", packet)
}
}
```

5. 블록 동기화 과정

downloader.Downloader 구조체

블록체인 데이터 동기화를 담당하는 구조체입니다.

```
type Downloader struct {
mode atomic.Uint32 // 동기화 모드 (FullSync, SnapSync)
// 동기화 상태
synchronising atomic.Bool
notified atomic.Bool
committed atomic.Bool
// 체인 관련
blockchain BlockChain
lightchain LightChain
stateDb ethdb.Database
// 피어 관리
peers *peerSet
// 큐 관리
queue *queue
// 이벤트 처리
quitCh  chan  struct{}
quitLock sync.Mutex
// 통계
syncStatsChainOrigin  uint64
syncStatsChainHeight  uint64
```

```
syncStatsState stateSyncStats

syncStatsLock sync.RWMutex

}
```

동기화 시작

```
func (d *Downloader) synchronise(id  string, hash common.Hash, td, ttd *big.Int,

// 동기화 모드 설정

d.mode.Store(uint32(mode))

// 피어 선택

p := d.peers.Peer(id)

if  p == nil {

return  errUnknownPeer

}

// 동기화 실행

return  d.syncWithPeer(p, hash, td, ttd)

}
```

코드 예시

P2P 서버 생성 및 시작

```
package main
```

```go
import (
"crypto/ecdsa"
"net"
"github.com/ethereum/go-ethereum/crypto"
"github.com/ethereum/go-ethereum/p2p"
"github.com/ethereum/go-ethereum/p2p/enode"
)

func  createP2PServer() (*p2p.Server, error) {
// 개인키 생성
key, err := crypto.GenerateKey()
if  err != nil {
return  nil, err
}
// 서버 설정
config := p2p.Config{
PrivateKey: key,
Name: "geth-node",
ListenAddr: ":30303",
MaxPeers: 50,
MaxPendingPeers: 10,
DiscoveryV4: true,
DiscoveryV5: true,
```

```
Protocols: []p2p.Protocol{}, // 프로토콜 추가
}
// 서버 생성
server := &p2p.Server{Config: config}
// 서버 시작
if err := server.Start(); err != nil {
return nil, err
}
return server, nil
}
```

피어 발견 사용 예시

```
func discoverPeers(server *p2p.Server) {
// UDPv4 디스커버리 인스턴스 가져오기
disc := server.DiscoveryV4()
if disc == nil {
return
}
// 랜덤 노드 검색
iterator := disc.RandomNodes()
defer iterator.Close()
for iterator.Next() {
```

```
node := iterator.Node()
fmt.Printf("Found peer: %s\n", node.String())
// 피어에 연결 시도
server.AddPeer(node)
}
}
```

프로토콜 핸들러 구현

```
type MyProtocol struct {
peer *p2p.Peer
}

func (mp *MyProtocol) Run(peer *p2p.Peer, rw p2p.MsgReadWriter) error {
mp.peer = peer
for {
msg, err := rw.ReadMsg()
if err != nil {
return err
}
switch msg.Code {
case 0x01: // 사용자 정의 메시지 타입
if err := mp.handleCustomMessage(msg); err != nil {
```

```go
return err
}
default:
msg.Discard()
}
}
}

func (mp *MyProtocol) handleCustomMessage(msg p2p.Msg) error {
// 메시지 처리 로직
defer msg.Discard()
var data []byte
if err := msg.Decode(&data); err != nil {
return err
}
fmt.Printf("Received custom message from %s: %x\n", mp.peer.ID(), data)
return nil
}
```

최적화 및 성능

연결 관리 최적화

1. **연결 제한**: `MaxPeers` 와 `MaxPendingPeers` 설정으로 리소스 사용량 제어
2. **피어 품질 관리**: 불량 피어 자동 차단 및 신뢰할 수 있는 피어 우선 연결
3. **대역폭 관리**: 메시지 크기 제한 및 전송 속도 조절

디스커버리 최적화

1. **부트스트랩 노드**: 초기 연결을 위한 신뢰할 수 있는 노드 목록 유지
2. **DHT 테이블 관리**: Kademlia 테이블의 효율적인 업데이트 및 유지
3. **네트워크 제한**: `NetRestrict` 설정으로 특정 네트워크 대역만 허용

메시지 처리 최적화

```
const (
// 메시지 크기 제한
maxMessageSize = 10 * 1024 * 1024  // 10MB
// 동시 처리 제한
maxConcurrentRequests = 100
// 타임아웃 설정
handshakeTimeout = 5 * time.Second
responseTimeout = 30 * time.Second
)
```

디버깅 및 모니터링

로깅 포인트

1. **연결 이벤트**: 피어 연결/해제 로그
2. **핸드셰이크 과정**: RLPx 및 프로토콜 핸드셰이크 상태
3. **메시지 교환**: 송수신 메시지 추적
4. **에러 상황**: 연결 실패, 프로토콜 오류 등

메트릭 수집

```go
var (
// 연결 관련 메트릭
activePeerGauge = metrics.NewRegisteredGauge("p2p/peers", nil)
dialSuccessMeter = metrics.NewRegisteredMeter("p2p/dials/success", nil)
dialErrorMeter = metrics.NewRegisteredMeter("p2p/dials/error", nil)
// 메시지 관련 메트릭
ingressTrafficMeter = metrics.NewRegisteredMeter("p2p/ingress", nil)
egressTrafficMeter = metrics.NewRegisteredMeter("p2p/egress", nil)
)
```

디버깅 도구

1. **피어 정보 조회**:

```go
func (srv *Server) NodeInfo() *NodeInfo {
node := srv.Self()
info := &NodeInfo{
Name: srv.Name,
```

```
Enode: node.URLv4(),
ID: node.ID().String(),
IP: node.IPAddr().String(),
ListenAddr: srv.ListenAddr,
Protocols: make(map[string]interface{}),
}
info.Ports.Discovery = node.UDP()
info.Ports.Listener = node.TCP()
return info
}
```

2. **연결 상태 모니터링**:

```
func (srv *Server) Peers() []*Peer {
var ps []*Peer
srv.doPeerOp(func(peers map[enode.ID]*Peer) {
for _, p := range peers {
ps = append(ps, p)
}
})
return ps
}
```

관련 컴포넌트

의존성 관계

- **crypto 패키지**: 암호화 키 관리 및 서명
- **enode 패키지**: 노드 식별 및 ENR 레코드 관리
- **rlpx 패키지**: 암호화된 전송 프로토콜
- **netutil 패키지**: 네트워크 유틸리티 함수

상호작용 방식

1. **eth 패키지와의 연동**: 이더리움 프로토콜 메시지 처리
2. **core 패키지와의 연동**: 블록체인 데이터 동기화
3. **txpool과의 연동**: 트랜잭션 전파 및 수집

P2P 네트워킹 시스템은 Geth의 분산 네트워크 참여를 가능하게 하는 핵심 인프라로, 안정적이고 효율적인 피어 간 통신을 통해 블록체인 네트워크의 무결성과 가용성을 보장합니다.